Modeling Human Activity States Using Hidden Markov Models

Name: Paulette Dushime

1. Background and Motivation

Smartphones carry motion sensors that record how a person moves, but the sensors never tell us the activity directly. They only give noisy streams of numbers. The true activity, such as walking, standing, jumping, or staying still, is hidden behind these measurements. This is exactly the structure that a Hidden Markov Model is designed for. My use case is remote patient activity monitoring. In many communities, including here in Rwanda, wearable medical devices are expensive like smart watches, but most families have access to a smartphone. If a phone can recognize whether an elderly person is walking, standing, or suddenly not moving at all, it can support fall detection and daily health tracking at almost no extra cost. In this project I collected my own motion data, built a Gaussian Hidden Markov Model completely from scratch in Python, trained it with the Baum-Welch algorithm, decoded activities with the Viterbi algorithm, and evaluated it on unseen recordings. Below we are going to explore how data was collected, the pre-processing and many others.

2. Data Collection and Preprocessing

I collected all the data myself using the Sensor Logger app on my I phone smartphone, recording the accelerometer (x, y, z) and gyroscope (x, y, z) at the same time. The phone was held at waist level for standing, for walking I was holding the phone in my hand, and finally placed flat on a table for the still activity and for jumping it was the same which was holding the phone in my hand. I recorded four activities: standing (about 2 minutes, phone held in my hand), walking (about 2 minutes at a steady pace), jumping (four separate clips of roughly 30 seconds each, about 2 minutes in total), and still (about 2 minutes with the phone untouched on a table). I also recorded three continuous mixed sessions where I moved between activities in one recording. One mixed session was used only for training, and two mixed sessions with known activity timings were kept as unseen test data.

My phone recorded both sensors at a native rate of approximately 100 Hz.

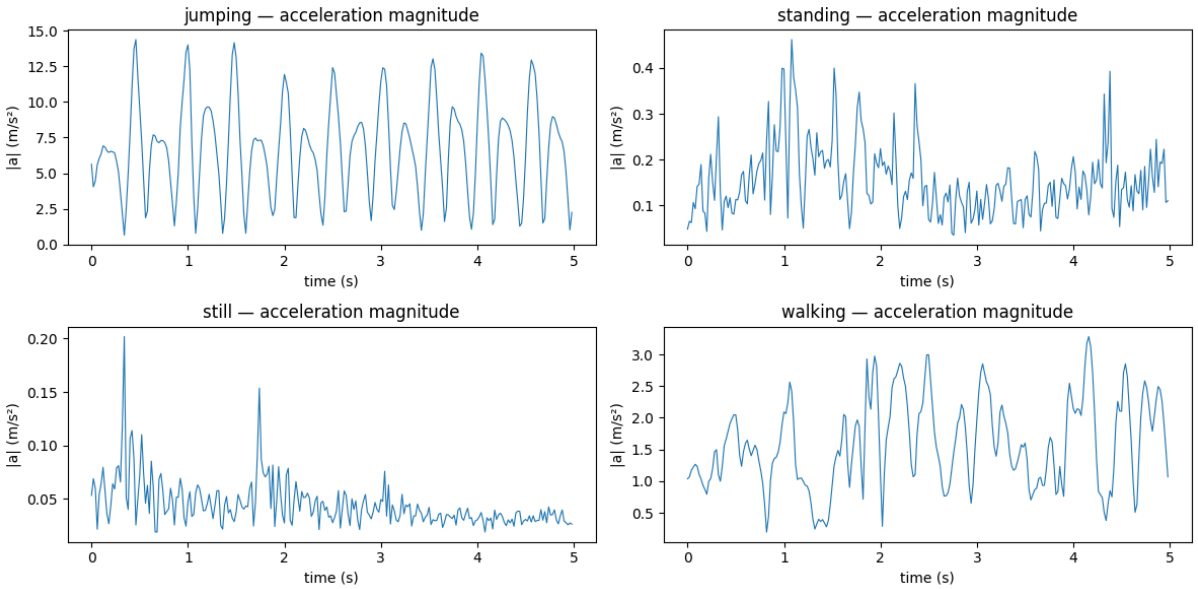
Member	Device sensors	Native sampling rate
Paulette Dushime	Accelerometer + Gyroscope	~100 Hz

Since different phones can record at different rates, all recordings were harmonized by resampling every axis onto a uniform 50 Hz time grid using linear interpolation. This guarantees that every window contains the same number of samples no matter which device produced the data. The gyroscope stream was also interpolated onto the accelerometer timestamps so both sensors share one time base.

Two cleaning steps were applied. First, 3 seconds were trimmed from the start and end of every recording, because pressing the record button and getting into position creates motion artifacts that do not belong to the activity. I discovered this by profiling my recordings second by second and finding movement spikes exactly at the start, end, and moments when I adjusted the phone. Second, each single activity recording was sliced into 5 second files that each inherit the activity label from the recording. This produced 79 training files and 15 unseen test files, all saved as CSV with a timestamp column.

For windowing, I used a window of 1 second which is 50 samples at 50 Hz with 50 percent overlap. The window size follows from the sampling rate and the physics of the activities: human walking happens at roughly 1.5 to 2.5 Hz and jumping at roughly 2 to 4 Hz, so a 1 second window always captures at least one full movement cycle. A 1 second window also gives the FFT a frequency resolution of 1 Hz, which is enough to separate the walking and jumping rhythms. The 50 percent overlap doubles the number of observations and smooths the observation sequence.

Figure 1: raw acceleration magnitude plots for the four activities



The raw signals already show four visibly different signatures. Jumping produces strong periodic spikes reaching about 15 m/s², walking shows a medium rhythmic pattern around 1 to 3 m/s², standing shows only small hand tremor below 0.5 m/s², and still is almost flat below 0.1 m/s².

3. HMM Setup and Implementation

The model components were defined as follows for my problem.

The hidden states Z are the four activities which are standing, walking, jumping, and still without moving. These are hidden because the phone never observes the activity itself, only its effect on the sensors.

The observations X are 7-dimensional feature vectors computed from each 1 second window. I extracted five time-domain features and two frequency domain features derived from the FFT.

Feature	Domain	Why it helps separate the activities
Mean of acceleration magnitude	Time	Overall motion level of the window
Log of standard deviation of acceleration magnitude	Time	Still is near zero, standing shows small tremor,

		walking is medium, jumping is large
Log of RMS of acceleration magnitude	Time	Measures movement intensity
Log of Signal Magnitude Area (SMA)	Time	A classic activity recognition intensity measure across all three axes
Log of standard deviation of gyroscope magnitude	Time	Rotation separates a phone held in a hand from a phone resting on a surface
Dominant frequency (FFT)	Frequency	Walking is about 1.5 to 2.5 Hz while jumping is about 2 to 4 Hz
Log of spectral energy (FFT)	Frequency	Total periodic power, which is near zero for quiet activities

The intensity features use a log transform because jumping produces values that are enormous compared to the quiet activities. Without the log, the difference between still and standing would be squashed into almost nothing after normalization. All features were then normalized using Z-score normalization, where the mean and standard deviation were computed on the training set only and applied unchanged to the test set. This prevents information from the test data leaking into training. Z-score was chosen because the features live on very different scales, for example dominant frequency in Hz versus spectral energy, and Gaussian emissions would otherwise be dominated by the largest scale feature.

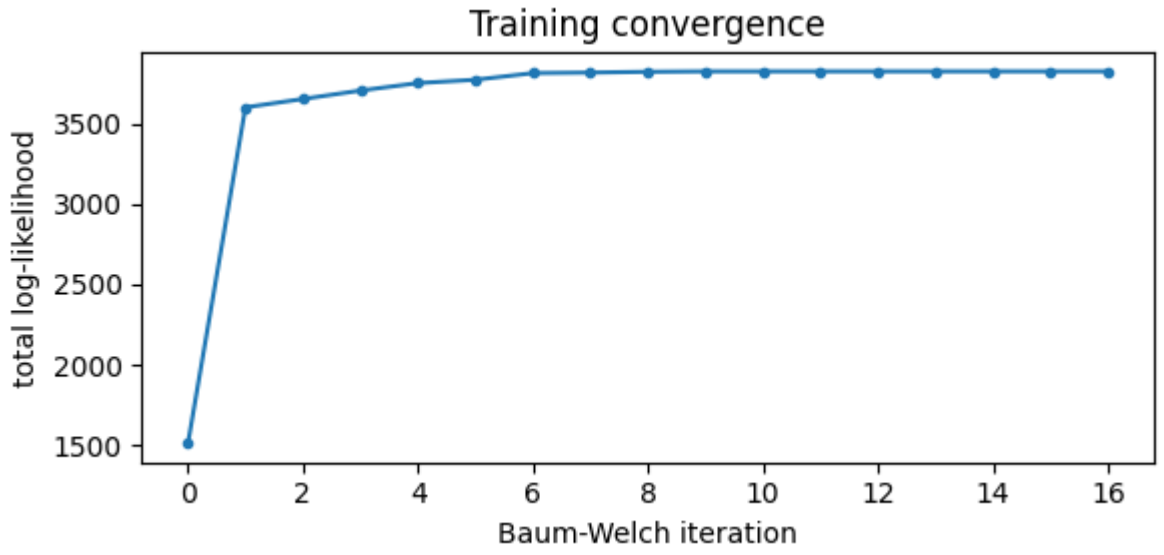
The transition matrix A is a 4 by 4 matrix giving the probability of moving from one activity to another between consecutive windows. The emission model B gives the probability of observing a feature vector given the hidden activity. Because my observations are continuous vectors rather than discrete symbols, B is a probability density: each hidden state has a Gaussian distribution with its own mean vector and diagonal covariance over the 7 features. The initial state probabilities π give the chance of each activity at the first window of a sequence.

I implemented the full model from scratch with numpy, without using hmmlearn. All the recursions run in log space using the log-sum-exp trick, because multiplying hundreds of probabilities together underflows floating point numbers. The Forward and Backward algorithms compute the state posteriors, and the Baum-Welch algorithm performs Expectation Maximization over all training sequences: in the E step it computes gamma which is the probability of being in each state at each time and xi which is the expected transition counts, and in the M step it re-estimates π, A, and the Gaussian means and variances from these soft counts. Training stops using a proper convergence check which is when the absolute change in total log likelihood falls below 1e-4. A small variance floor of 1e-3 keeps the Gaussians numerically stable.

Since I know which activity each single activity training file contains, I initialized each hidden state's Gaussian at that activity's mean feature vector, and then let Baum-Welch refine all parameters jointly, including on the unlabelled mixed training session. This informed initialization keeps hidden state k aligned with activity k and avoids poor local optima, which is a known weakness of EM. After training, hidden states were mapped to activity names by decoding the labelled training files with Viterbi and taking a majority vote.

The Viterbi algorithm was implemented in log space with backpointers. At every time step it keeps, for each state, the probability of the best path ending in that state, and remembers which previous state produced it. At the end it backtracks through the pointers to recover the single most likely activity sequence.

Figure 2: training convergence plot



Training converged at iteration 16, with the log-likelihood rising steeply in the first iterations and then flattening, which shows that EM behaved correctly and the convergence criterion worked.

4. Results and Interpretation

Figure 3: transition matrix and emission means heatmaps

The learned transition matrix has values of 0.97 to 0.99 on the diagonal. This reflects realistic behavior, the activities persist across consecutive half second steps, and a person does not switch activities every window. The small off diagonal values correspond to the real transitions present in the mixed training session. The emission means heatmap is also interpretable where jumping is strongly positive on every intensity feature, still is strongly negative, and standing sits just above still on all features because of natural hand tremor. Still shows a higher dominant frequency value, which makes sense because pure sensor noise has a random dominant frequency rather than a movement rhythm.

The model was evaluated on 15 unseen test files that were never used in training which were instead kept out for 5 second slices of every activity, plus two continuous mixed sessions with known activity timings recorded in separate sessions. Test set 2 was recorded on a different day from most of the training data.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Standing	93	0.882	0.987	0.886
Walking	223	0.978	0.804	0.886
Jumping	121	0.686	1.000	0.886
Still	36	1.000	1.000	0.886

Figure 4: confusion matrix

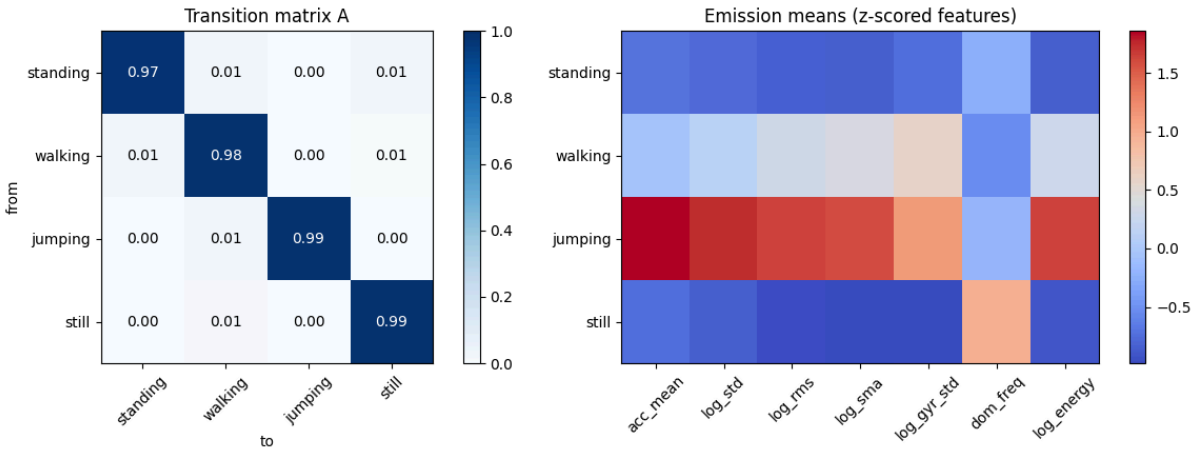
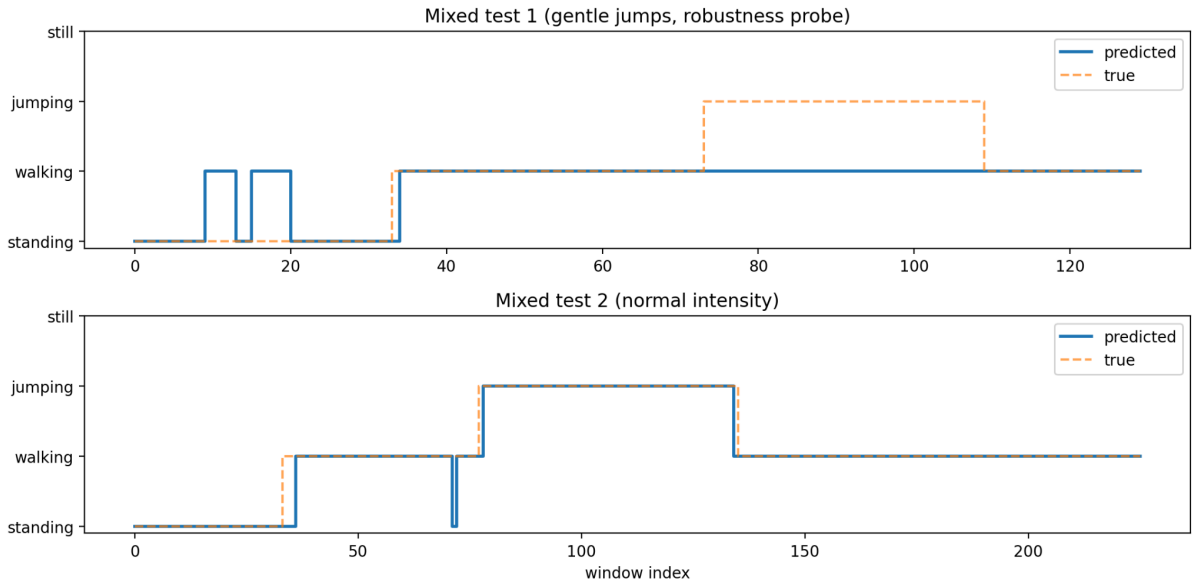


Figure 5: decoded sequence plots for mixed test 1 and mixed test 2



The overall accuracy across all unseen data was 88.6 percent, but the per file breakdown tells a more interesting story. On the second mixed session, where all activities were performed at normal intensity, the model reached 97.3 percent accuracy with jumping sensitivity of 0.966, and the decoded sequence follows the true sequence almost perfectly, including the transitions between activities. All held out single activity slices for jumping, walking, and still decoded correctly as well. The lower combined jumping sensitivity comes entirely from the first mixed session, which I kept deliberately as a robustness test and in that session I jumped very gently because my legs were tired, and the model classified those windows as walking since for it makes sense cause slow jumps can mean that i was walking. The confusion matrix shows this clearly, since the only large error block is jumping predicted as walking.

5. Discussion and Conclusion

The easiest activities to distinguish were jumping and still, which sit at the two extremes of motion intensity, and both reached perfect scores on their held out slices. The hardest pair was still versus standing, and this taught me the most important lesson of the project. My first standing recording produced a model that merged still and standing into one state. Since my first recording while studying I just put my phone on the table which is also what i did while still. When I profiled the recording second by second, I found the motion level was 0.005 to 0.01, which is the stillness of a phone resting on a surface, not a phone in a human hand. The phone had effectively been at rest, so the two classes were physically identical in the data. After re-recording standing with the phone held naturally in my hand, the natural hand tremor where motion level was around 0.05 to 0.15 separated the classes, and still sensitivity jumped from 0.25 to 1.00. This showed me that data collection protocol matters as much as the model itself.

The transition probabilities reflect realistic behavior patterns. The strong diagonal says that activities persist over time, and the model uses this to smooth out single noisy windows where one strange window in the middle of a walk does not flip the prediction, because the transition cost of leaving and re-entering a state outweighs one window of weak emission evidence.

Sensor noise and the recording process affected the model in concrete ways. Pressing the start and stop buttons created motion spikes at the edges of recordings, which is why I trimmed 3 seconds from each end. Resampling from 100 Hz to 50 Hz smooths some high frequency content, but the activity rhythms of interest are below 5 Hz, so no useful information was lost while the data became uniform and comparable.

The most valuable finding came from the first mixed test session. The gentle jumps had an acceleration intensity of about 1.5, which falls inside the walking distribution of about 1.1, while my training jumps averaged about 3.6. Even the frequency features overlapped, with gentle jumps at 2.8 Hz plus or minus 1.6 against walking at 1.9 Hz plus or minus 1.1. This means the model learned intensity specific signatures of my energetic jumping rather than an abstract concept of jumping, which is a form of distribution shift. It is also a realistic warning for the health monitoring use case, because elderly users move more gently than the person who collects the training data.

Several improvements follow directly from these findings. Training data should cover multiple movement intensities for each activity, and ideally multiple participants, phone positions, and environments. Full covariance Gaussians could capture correlations between features that the diagonal model ignores. Additional sensors such as the magnetometer or

barometer could add information that intensity features cannot provide. Finally, running Baum-Welch from several random initializations and keeping the best likelihood would reduce the risk of local optima when labelled initialization is not available.

In conclusion, I built a complete human activity recognition pipeline from raw smartphone sensors to decoded activity sequences, with every algorithm implemented from scratch. The model recognizes normal intensity activities on unseen data with about 97 percent accuracy, its learned parameters are physically interpretable, and its one systematic failure mode was found, explained, and traced to a concrete cause in the training data. Both the strengths and the failure taught me the same lesson that in applied machine learning, understanding your data is as important as understanding your algorithm.